

- ✓ Generative AI & Transformers Discovery Project
- ✓ Discovery Phase: Setup and Initialization

First, we need to install the required libraries: `transformers` for accessing HuggingFace models and `torch` as its backend. We'll use `pip` for installation.

```
%%capture
!pip install transformers torch
```

Now that the libraries are installed, we can import `pipeline` from the `transformers` library and initialize the three distinct HuggingFace pipelines: `sentiment-analysis`, `text-generation`, and `summarization`.

```
from transformers import pipeline

# Initialize the sentiment-analysis pipeline
sentiment_pipeline = pipeline('sentiment-analysis')
print('Sentiment Analysis Pipeline Initialized:')
print(sentiment_pipeline.model)
print(sentiment_pipeline.tokenizer)

# Initialize the text-generation pipeline
text_generation_pipeline = pipeline('text-generation')
print('\nText Generation Pipeline Initialized:')
print(text_generation_pipeline.model)
print(text_generation_pipeline.tokenizer)
```



```

[transformers] No model was supplied, defaulted to distilbert/distilbert-
Using a pipeline without specifying a model name and revision in producti
Loading weights: 100% 104/104 [00:00<00:00, 2448.35it/s]

[transformers] No model was supplied, defaulted to HuggingFaceTB/SmolLM3-
Using a pipeline without specifying a model name and revision in producti
Sentiment Analysis Pipeline Initialized:
DistilBertForSequenceClassification(
  (distilbert): DistilBertModel(
    (embeddings): Embeddings(
      (word_embeddings): Embedding(30522, 768, padding_idx=0)
      (position_embeddings): Embedding(512, 768)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (transformer): Transformer(
      (layer): ModuleList(
        (0-5): 6 x TransformerBlock(
          (attention): DistilBertSelfAttention(
            (q_lin): Linear(in_features=768, out_features=768, bias=True)
            (k_lin): Linear(in_features=768, out_features=768, bias=True)
            (v_lin): Linear(in_features=768, out_features=768, bias=True)
            (out_lin): Linear(in_features=768, out_features=768, bias=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
          (sa_layer_norm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
          (ffn): FFN(
            (dropout): Dropout(p=0.1, inplace=False)
            (lin1): Linear(in_features=768, out_features=3072, bias=True)
            (lin2): Linear(in_features=3072, out_features=768, bias=True)
            (activation): GELUActivation()
          )
          (output_layer_norm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
        )
      )
    )
  )
  (pre_classifier): Linear(in_features=768, out_features=768, bias=True)
  (classifier): Linear(in_features=768, out_features=2, bias=True)
  (dropout): Dropout(p=0.2, inplace=False)
)
BertTokenizer(name_or_path='distilbert/distilbert-base-uncased-finetuned-
0: AddedToken("[PAD]", rstrip=False, lstrip=False, single_word=False,
100: AddedToken("[UNK]", rstrip=False, lstrip=False, single_word=False)
101: AddedToken("[CLS]", rstrip=False, lstrip=False, single_word=False)
102: AddedToken("[SEP]", rstrip=False, lstrip=False, single_word=False)
103: AddedToken("[MASK]", rstrip=False, lstrip=False, single_word=False)
})
Loading weights: 100% 326/326 [00:00<00:00, 3113.92it/s]

```

Text Generation Pipeline Initialized:

```

SmolLM3ForCausalLM(
  (model): SmolLM3Model(
    (embed_tokens): Embedding(128256, 2048, padding_idx=128004)
    (layers): ModuleList(
      (0-25): 26 x SmolLM3DecoderLayer(

```

```

    (0-33): 30 x SmolLM3DecoderLayer(
      (self_attn): SmolLM3Attention(
        (q_proj): Linear(in_features=2048, out_features=2048, bias=False)
        (k_proj): Linear(in_features=2048, out_features=512, bias=False)
        (v_proj): Linear(in_features=2048, out_features=512, bias=False)
        (o_proj): Linear(in_features=2048, out_features=2048, bias=False)
      )
      (mlp): SmolLM3MLP(
        (gate_proj): Linear(in_features=2048, out_features=11008, bias=False)
        (up_proj): Linear(in_features=2048, out_features=11008, bias=False)
        (down_proj): Linear(in_features=11008, out_features=2048, bias=False)
        (act_fn): SiLUActivation()
      )
      (input_layernorm): SmolLM3RMSNorm((2048,), eps=1e-06)
      (post_attention_layernorm): SmolLM3RMSNorm((2048,), eps=1e-06)
    )
  )
  (norm): SmolLM3RMSNorm((2048,), eps=1e-06)
  (rotary_emb): SmolLM3RotaryEmbedding()
)
(lm_head): Linear(in_features=2048, out_features=128256, bias=False)
)
TokenizersBackend(name_or_path='HuggingFaceTB/SmolLM3-3B', vocab_size=128256)
128000: AddedToken("<|begin_of_text|>", rstrip=False, lstrip=False, single_word=False)
128001: AddedToken("<|end_of_text|>", rstrip=False, lstrip=False, single_word=False)
128002: AddedToken("<think>", rstrip=False, lstrip=False, single_word=False)
128003: AddedToken("</think>", rstrip=False, lstrip=False, single_word=False)
128004: AddedToken("<|finetune_right_pad_id|>", rstrip=False, lstrip=False, single_word=False)
128005: AddedToken("<|reserved_special_token_2|>", rstrip=False, lstrip=False, single_word=False)
128006: AddedToken("<|start_header_id|>", rstrip=False, lstrip=False, single_word=False)
128007: AddedToken("<|end_header_id|>", rstrip=False, lstrip=False, single_word=False)
128008: AddedToken("<|eom_id|>", rstrip=False, lstrip=False, single_word=False)
128009: AddedToken("<|eot_id|>", rstrip=False, lstrip=False, single_word=False)
128010: AddedToken("<|python_tag|>", rstrip=False, lstrip=False, single_word=False)
128011: AddedToken("<|im_start|>", rstrip=False, lstrip=False, single_word=False)
128012: AddedToken("<|im_end|>", rstrip=False, lstrip=False, single_word=False)
128013: AddedToken("<tool_response>", rstrip=False, lstrip=False, single_word=False)
128014: AddedToken("</tool_response>", rstrip=False, lstrip=False, single_word=False)
128015: AddedToken("<tool_call>", rstrip=False, lstrip=False, single_word=False)
128016: AddedToken("</tool_call>", rstrip=False, lstrip=False, single_word=False)
128017: AddedToken("<code>", rstrip=False, lstrip=False, single_word=False)
128018: AddedToken("</code>", rstrip=False, lstrip=False, single_word=False)
128019: AddedToken("<|reserved_special_token_11|>", rstrip=False, lstrip=False, single_word=False)
128020: AddedToken("<|reserved_special_token_12|>", rstrip=False, lstrip=False, single_word=False)
128021: AddedToken("<|reserved_special_token_13|>", rstrip=False, lstrip=False, single_word=False)
128022: AddedToken("<|reserved_special_token_14|>", rstrip=False, lstrip=False, single_word=False)
128023: AddedToken("<|reserved_special_token_15|>", rstrip=False, lstrip=False, single_word=False)
128024: AddedToken("<|reserved_special_token_16|>", rstrip=False, lstrip=False, single_word=False)
128025: AddedToken("<|reserved_special_token_17|>", rstrip=False, lstrip=False, single_word=False)
128026: AddedToken("<|reserved_special_token_18|>", rstrip=False, lstrip=False, single_word=False)
128027: AddedToken("<|reserved_special_token_19|>", rstrip=False, lstrip=False, single_word=False)
128028: AddedToken("<|reserved_special_token_20|>", rstrip=False, lstrip=False, single_word=False)
128029: AddedToken("<|reserved_special_token_21|>", rstrip=False, lstrip=False, single_word=False)
128030: AddedToken("<|reserved_special_token_22|>", rstrip=False, lstrip=False, single_word=False)
128031: AddedToken("<|reserved_special_token_23|>", rstrip=False, lstrip=False, single_word=False)
128032: AddedToken("<|reserved_special_token_24|>", rstrip=False, lstrip=False, single_word=False)
128033: AddedToken("<|reserved_special_token_25|>", rstrip=False, lstrip=False, single_word=False)

```

[illegible]

[illegible]

[illegible]

```

128204: AddedToken("<|reserved_special_token_196|>", rstrip=False, lst
128205: AddedToken("<|reserved_special_token_197|>", rstrip=False, lst
128206: AddedToken("<|reserved_special_token_198|>", rstrip=False, lst
128207: AddedToken("<|reserved_special_token_199|>", rstrip=False, lst
128208: AddedToken("<|reserved_special_token_200|>", rstrip=False, lst
128209: AddedToken("<|reserved_special_token_201|>", rstrip=False, lst

```

To improve clarity and modularity, the initialization of each Hugging Face pipeline has been separated into its own cell. This makes it easier to understand individual components and manage their dependencies.

```

# Initialize the sentiment-analysis pipeline
sentiment_pipeline = pipeline('sentiment-analysis')
print('Sentiment Analysis Pipeline Initialized:')
print(sentiment_pipeline.model)
print(sentiment_pipeline.tokenizer)

```

```

128217: AddedToken("<|reserved_special_token_209|>", rstrip=False, lst
128218: AddedToken("<|reserved_special_token_210|>", rstrip=False, lst
128219: AddedToken("<|reserved_special_token_211|>", rstrip=False, lst
128220: AddedToken("<|reserved_special_token_212|>", rstrip=False, lst
128221: AddedToken("<|reserved_special_token_213|>", rstrip=False, lst
128222: AddedToken("<|reserved_special_token_214|>", rstrip=False, lst
128223: AddedToken("<|reserved_special_token_215|>", rstrip=False, lst
128224: AddedToken("<|reserved_special_token_216|>", rstrip=False, lst
128225: AddedToken("<|reserved_special_token_217|>", rstrip=False, lst
128226: AddedToken("<|reserved_special_token_218|>", rstrip=False, lst
128227: AddedToken("<|reserved_special_token_219|>", rstrip=False, lst
128228: AddedToken("<|reserved_special_token_220|>", rstrip=False, lst
128229: AddedToken("<|reserved_special_token_221|>", rstrip=False, lst
128230: AddedToken("<|reserved_special_token_222|>", rstrip=False, lst
128231: AddedToken("<|reserved_special_token_223|>", rstrip=False, lst
128232: AddedToken("<|reserved_special_token_224|>", rstrip=False, lst
128233: AddedToken("<|reserved_special_token_225|>", rstrip=False, lst
128234: AddedToken("<|reserved_special_token_226|>", rstrip=False, lst
128235: AddedToken("<|reserved_special_token_227|>", rstrip=False, lst
128236: AddedToken("<|reserved_special_token_228|>", rstrip=False, lst
128237: AddedToken("<|reserved_special_token_229|>", rstrip=False, lst
128238: AddedToken("<|reserved_special_token_230|>", rstrip=False, lst
128239: AddedToken("<|reserved_special_token_231|>", rstrip=False, lst
128240: AddedToken("<|reserved_special_token_232|>", rstrip=False, lst
128241: AddedToken("<|reserved_special_token_233|>", rstrip=False, lst
128242: AddedToken("<|reserved_special_token_234|>", rstrip=False, lst
128243: AddedToken("<|reserved_special_token_235|>", rstrip=False, lst
128244: AddedToken("<|reserved_special_token_236|>", rstrip=False, lst
128245: AddedToken("<|reserved_special_token_237|>", rstrip=False, lst
128246: AddedToken("<|reserved_special_token_238|>", rstrip=False, lst
128247: AddedToken("<|reserved_special_token_239|>", rstrip=False, lst
128248: AddedToken("<|reserved_special_token_240|>", rstrip=False, lst
128249: AddedToken("<|reserved_special_token_241|>", rstrip=False, lst
128250: AddedToken("<|reserved_special_token_242|>", rstrip=False, lst
128251: AddedToken("<|reserved_special_token_243|>", rstrip=False, lst
128252: AddedToken("<|reserved_special_token_244|>", rstrip=False, lst
128253: AddedToken("<|reserved_special_token_245|>", rstrip=False, lst
128254: AddedToken("<|reserved_special_token_246|>", rstrip=False, lst
128255: AddedToken("<|reserved_special_token_247|>", rstrip=False, lst
}

```

```

[transformers] No model was supplied, defaulted to distilbert/distilbert-
Using a pipeline without specifying a model name and revision in producti
Loading weights: 100% 104/104 [00:00<00:00, 3414.25it/s]
Sentiment Analysis Pipeline Initialized:
DistilBertForSequenceClassification(
  (distilbert): DistilBertModel(
    (embeddings): Embeddings(
      (word_embeddings): Embedding(30522, 768, padding_idx=0)
      (position_embeddings): Embedding(512, 768)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (transformer): Transformer(
      (layer): ModuleList(
        (0-5): 6 x TransformerBlock(
          (attention): DistilBertSelfAttention(
            (q_lin): Linear(in_features=768, out_features=768, bias=True)
            (k_lin): Linear(in_features=768, out_features=768, bias=True)
            (v_lin): Linear(in_features=768, out_features=768, bias=True)
            (out_lin): Linear(in_features=768, out_features=768, bias=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
          (sa_layer_norm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
          (ffn): FFN(
            (dropout): Dropout(p=0.1, inplace=False)
            (lin1): Linear(in_features=768, out_features=3072, bias=True)
            (lin2): Linear(in_features=3072, out_features=768, bias=True)
            (activation): GELUActivation()
          )
          (output_layer_norm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
        )
      )
    )
  )
  (pre_classifier): Linear(in_features=768, out_features=768, bias=True)
  (classifier): Linear(in_features=768, out_features=2, bias=True)
  (dropout): Dropout(p=0.2, inplace=False)
)
BertTokenizer(name_or_path='distilbert/distilbert-base-uncased-finetuned-sst-2-v1')
  0: AddedToken("[PAD]", rstrip=False, lstrip=False, single_word=False,
  100: AddedToken("[UNK]", rstrip=False, lstrip=False, single_word=False,
  101: AddedToken("[CLS]", rstrip=False, lstrip=False, single_word=False,
  102: AddedToken("[SEP]", rstrip=False, lstrip=False, single_word=False,
  103: AddedToken("[MASK]", rstrip=False, lstrip=False, single_word=False,
  })

```

```

# Initialize the text-generation pipeline
text_generation_pipeline = pipeline('text-generation')
print('\nText Generation Pipeline Initialized:')
print(text_generation_pipeline.model)
print(text_generation_pipeline.tokenizer)

```


[transformers] No model was supplied, defaulted to HuggingFaceTB/SmolLM3-
Using a pipeline without specifying a model name and revision in producti

Loading weights: 100%

326/326 [00:00<00:00, 3739.73it/s]

Text Generation Pipeline Initialized:

SmolLM3ForCausalLM(
(model): SmolLM3Model(
(embed_tokens): Embedding(128256, 2048, padding_idx=128004)
(layers): ModuleList(
(0-35): 36 x SmolLM3DecoderLayer(
(self_attn): SmolLM3Attention(
(q_proj): Linear(in_features=2048, out_features=2048, bias=False)
(k_proj): Linear(in_features=2048, out_features=512, bias=False)
(v_proj): Linear(in_features=2048, out_features=512, bias=False)
(o_proj): Linear(in_features=2048, out_features=2048, bias=False)
)
(mlp): SmolLM3MLP(
(gate_proj): Linear(in_features=2048, out_features=11008, bias=False)
(up_proj): Linear(in_features=2048, out_features=11008, bias=False)
(down_proj): Linear(in_features=11008, out_features=2048, bias=False)
(act_fn): SiLUActivation()
)
(input_layernorm): SmolLM3RMSNorm((2048,), eps=1e-06)
(post_attention_layernorm): SmolLM3RMSNorm((2048,), eps=1e-06)
)
)
(norm): SmolLM3RMSNorm((2048,), eps=1e-06)
(rotary_emb): SmolLM3RotaryEmbedding()
)
(lm_head): Linear(in_features=2048, out_features=128256, bias=False)
)

TokenizersBackend(name_or_path='HuggingFaceTB/SmolLM3-3B', vocab_size=128
128000: AddedToken("<|begin_of_text|>", rstrip=False, lstrip=False, si
128001: AddedToken("<|end_of_text|>", rstrip=False, lstrip=False, sing
128002: AddedToken("<think>", rstrip=False, lstrip=False, single_word=
128003: AddedToken("</think>", rstrip=False, lstrip=False, single_wor
128004: AddedToken("<|finetune_right_pad_id|>", rstrip=False, lstrip=F
128005: AddedToken("<|reserved_special_token_2|>", rstrip=False, lstri
128006: AddedToken("<|start_header_id|>", rstrip=False, lstrip=False,
128007: AddedToken("<|end_header_id|>", rstrip=False, lstrip=False, si
128008: AddedToken("<|eom_id|>", rstrip=False, lstrip=False, single_wc
128009: AddedToken("<|eot_id|>", rstrip=False, lstrip=False, single_wc
128010: AddedToken("<|python_tag|>", rstrip=False, lstrip=False, singl
128011: AddedToken("<|im_start|>", rstrip=False, lstrip=False, single_
128012: AddedToken("<|im_end|>", rstrip=False, lstrip=False, single_wc
128013: AddedToken("<tool_response>", rstrip=False, lstrip=False, sing
128014: AddedToken("</tool_response>", rstrip=False, lstrip=False, sir
128015: AddedToken("<tool_call>", rstrip=False, lstrip=False, single_w
128016: AddedToken("</tool_call>", rstrip=False, lstrip=False, single_
128017: AddedToken("<code>", rstrip=False, lstrip=False, single_word=F
128018: AddedToken("</code>", rstrip=False, lstrip=False, single_word=
128019: AddedToken("<|reserved_special_token_11|>", rstrip=False, lstr
128020: AddedToken("<|reserved_special_token_12|>", rstrip=False, lstr
128021: AddedToken("<|reserved_special_token_13|>", rstrip=False, lstr
128022: AddedToken("<|reserved_special_token_14|>", rstrip=False, lstr
128023: AddedToken("<|reserved_special_token_15|>", rstrip=False, lstr

[illegible]

[illegible]

[illegible]

[illegible]

```

128249: Added token( <|reserved_special_token_241|> , lstrip=False, lstr
# Initialize the summarization pipeline with a specific model for robust
summarization_pipeline = pipeline('text2text-generation', model='sshleif
print('\nSummarization Pipeline Initialized:')
print(summarization_pipeline.model)
print(summarization_pipeline.tokenizer)
'''
-----
---
KeyError                                Traceback (most recent call
last)
/tmp/ipykernel_2670/4087623082.py in <cell line: 0>()
      1 # Initialize the summarization pipeline with a specific model
for robustness
----> 2 summarization_pipeline = pipeline('text2text-generation',
model='sshleifer/distilbart-cnn-12-6')
      3 print('\nSummarization Pipeline Initialized:')
      4 print(summarization_pipeline.model)
      5 print(summarization_pipeline.tokenizer)

----- 2 frames -----
/usr/local/lib/python3.12/dist-packages/transformers/pipelines/base.py
in check_task(self, task)
    1338         return task, targeted_task, None
    1339
-> 1340         raise KeyError(f"Unknown task {task}, available tasks
are {self.get_supported_tasks()}")
    1341
    1342     def register_pipeline(

```

You can see the default model architectures loaded for each pipeline above. For example, `sentiment-analysis` often uses a BERT-like model, `text-generation` uses a GPT-like model (like GPT-2), and `summarization` might use models like BART or T5. Each pipeline comes with its corresponding tokenizer.

✓ Technical Phase: Implementation

Audit Task: Sentiment Analysis

We will now run an 'Audit' by processing 5 test sentences through the `sentiment-analysis` pipeline. This includes a sarcastic example to evaluate the model's nuance detection capabilities.

```

test_sentences = [
    "I love this product, it's absolutely fantastic!",
    "This service is terrible, I'm extremely disappointed.",
    "The weather today is just delightful, isn't it? (said during a heav

```

```

        "It was okay, nothing special.",
        "I'm so excited to get started on this project!"
    ]

    print("--- Sentiment Analysis Audit Results ---")
    for i, sentence in enumerate(test_sentences):
        result = sentiment_pipeline(sentence)
        print(f"\nSentence {i+1}: '{sentence}'")
        print(f"Sentiment: {result[0]['label']} (Score: {result[0]['score']}:

```

✓ Creator Task: Text Generation

Now, we will execute the 'Creator' task using the `text-generation` pipeline. We'll use a domain-specific prompt related to customer support or a similar field and constrain the output to `max_length=50` to observe the model's ability to generate concise text.

```

prompt = "In the context of customer support, a common issue is user pas

print("--- Text Generation Creator Task ---")
# Fixing the length constraint: use max_new_tokens to ensure the desired
generated_text = text_generation_pipeline(prompt, max_new_tokens=50, num

print(f"Prompt: '{prompt}'")
print(f"Generated Text: '{generated_text[0]['generated_text']}'")

```

```

print('\n--- Text Generation Creator Task (Experimenting with num_return

# Experiment with num_return_sequences
# Let's generate 3 different sequences to observe variety
num_sequences_to_generate = 3

generated_texts_experiment = text_generation_pipeline(
    prompt,
    max_new_tokens=50,
    num_return_sequences=num_sequences_to_generate,
    clean_up_tokenization_spaces=True
)

print(f"Prompt: '{prompt}'")
for i, result in enumerate(generated_texts_experiment):
    print(f"Generated Text {i+1}: '{result['generated_text']}'")

```

✓ Executive Summary Task: Summarization

For the 'Executive Summary' task, we will feed a 100-200 word news article into the `summarization` pipeline and evaluate the 20-30 word output for coherence and key point retention.

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
import warnings

# Suppress UserWarning about HF_TOKEN if it's purely informational
warnings.filterwarnings("ignore", message="You are not authenticated wit")
warnings.filterwarnings("ignore", message="You are sending unauthenticat")

model_name = 'sshleifer/distilbart-cnn-12-6'

# Explicitly load the summarization tokenizer and model
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

news_article = """Recent advancements in artificial intelligence have un

print("--- Summarization Executive Summary Task ---")

# Prepare the input for the model
inputs = tokenizer(news_article, return_tensors='pt', max_length=1024, t

# Generate the summary using the model's generate method
# We need to explicitly set min_length and max_length for the desired 20
# A common heuristic is 1 token ~ 0.75 words, so 20-30 words would be ar
summary_ids = model.generate(inputs['input_ids'], min_length=20, max_len
summary_text = tokenizer.decode(summary_ids[0], skip_special_tokens=True

print(f"\nOriginal Article (Word Count: {len(news_article.split())}): \n{
print(f"\nGenerated Summary (Word Count: {len(summary_text.split())}): \n{
```

```

tokenizer_config.json: 100%                26.0/26.0 [00:00<00:00, 1.56kB/s]
vocab.json: 100%                          899k/899k [00:00<00:00, 25.8MB/s]
merges.txt: 100%                          456k/456k [00:00<00:00, 17.8MB/s]
pytorch_model.bin: 100%                   1.22G/1.22G [00:18<00:00, 69.5MB/s]
[transformers] Please make sure the generation config includes `forced_bos_token_id`
model.safetensors: 100%                   1.22G/1.22G [00:23<00:00, 51.7MB/s]
Loading weights: 100%                     358/358 [00:00<00:00, 5513.27it/s]
--- Summarization Executive Summary Task ---

Original Article (Word Count: 152):
Recent advancements in artificial intelligence have unveiled new possibilities in the field of machine learning.

Generated Summary (Word Count: 35):
Researchers at leading tech institutions have developed a novel AI model that can generate human-like text.

```

✓ Final Validation Check

✓ Sentiment Analysis Validation

```

test_sentences = [
    "I love this product, it's absolutely fantastic!",
    "This service is terrible, I'm extremely disappointed.",
    "The weather today is just delightful, isn't it? (said during a heavy rain)",
    "It was okay, nothing special.",
    "I'm so excited to get started on this project!"
]

# Re-running Sentiment Analysis to ensure full output for README
print("--- Sentiment Analysis Final Validation Results ---")

for i, sentence in enumerate(test_sentences):
    result = sentiment_pipeline(sentence)
    # The output from the pipeline is a list of dicts, e.g., [{'label': 'positive', 'score': 0.9999}]
    sentiment_label = result[0]['label']
    sentiment_score = result[0]['score']
    print(f"\nSentence {i+1}: '{sentence}'")
    print(f"Sentiment: {sentiment_label} (Score: {sentiment_score:.4f})"

--- Sentiment Analysis Final Validation Results ---

Sentence 1: 'I love this product, it's absolutely fantastic!'
Sentiment: POSITIVE (Score: 0.9999)

```

Sentence 2: 'This service is terrible, I'm extremely disappointed.'
Sentiment: NEGATIVE (Score: 0.9997)

Sentence 3: 'The weather today is just delightful, isn't it? (said during
Sentiment: POSITIVE (Score: 0.9998)

Sentence 4: 'It was okay, nothing special.'
Sentiment: NEGATIVE (Score: 0.9821)

Sentence 5: 'I'm so excited to get started on this project!'
Sentiment: POSITIVE (Score: 0.9997)

✓ Text Generation Validation

```
prompt = "In the context of customer support, a common issue is user pas
num_sequences_to_generate = 3

# Re-running Text Generation with num_return_sequences to ensure full ou
print('\n--- Text Generation Final Validation (Experimenting with num_re

generated_texts_final = text_generation_pipeline(
    prompt,
    max_new_tokens=50,
    num_return_sequences=num_sequences_to_generate,
    clean_up_tokenization_spaces=True
)

print(f"Prompt: '{prompt}'")
for i, result in enumerate(generated_texts_final):
    print(f"Generated Text {i+1}: '{result['generated_text']}'")

# Placeholder for detailed discussion on coherence - will be integrated
print("\n--- Discussion on coherence (to be added to README) ---")
print("The generated texts demonstrate varying degrees of coherence and
```

```
[transformers] Passing `generation_config` together with generation-relat
[transformers] Both `max_new_tokens` (=50) and `max_length` (=20) seem to
```

```
--- Text Generation Final Validation (Experimenting with num_return_seque
[transformers] Ignoring clean_up_tokenization_spaces=True for BPE tokeniz
Prompt: 'In the context of customer support, a common issue is user passw
Generated Text 1: 'In the context of customer support, a common issue is
```

```
To streamline this process, a new approach could'
Generated Text 2: 'In the context of customer support, a common issue is
```

```
However, if the customer has already requested a password reset in the pa
Generated Text 3: 'In the context of customer support, a common issue is
```

```
Here is a sample Python code that can be used to implement a basic passwo
```

```
--- Discussion on coherence (to be added to README) ---  
The generated texts demonstrate varying degrees of coherence and relevance
```

✓ Summarization Validation

```
# Re-running Summarization to confirm word count comparison for README  
print("--- Summarization Final Validation Results ---")  
  
# Ensure news_article, tokenizer, and model are defined from cell bd6b9c  
inputs = tokenizer(news_article, return_tensors='pt', max_length=1024, truncation=True)  
summary_ids = model.generate(inputs['input_ids'], min_length=20, max_length=1024, num_return_sequences=1)  
summary_text_final = tokenizer.decode(summary_ids[0], skip_special_tokens=True)  
  
original_word_count = len(news_article.split())  
summary_word_count = len(summary_text_final.split())  
  
print(f"\nOriginal Article (Word Count: {original_word_count}): \n{news_article}")  
print(f"\nGenerated Summary (Word Count: {summary_word_count}): \n{summary_text_final}")  
print(f"Summary Length Comparison: Original has {original_word_count} words, Summary has {summary_word_count} words.")  
  
--- Summarization Final Validation Results ---  
  
Original Article (Word Count: 152):  
Recent advancements in artificial intelligence have unveiled new possibilities in the field of machine learning.  
  
Generated Summary (Word Count: 35):  
Researchers at leading tech institutions have developed a novel AI model that can generate human-like text.  
Summary Length Comparison: Original has 152 words, Summary has 35 words.
```

✓ Action Phase

Pre-training vs. Fine-tuning Explanation

In this section, we will explain the conceptual difference between Pre-training (learning general language patterns) and Fine-tuning (adapting to specific tasks) in your own words, without using jargon.

Pre-training vs. Fine-tuning: An Analogy

Imagine you want to teach someone to become an expert chef.

- **Pre-training** is like sending them to a general culinary school. They learn fundamental cooking techniques, understand different ingredients, and get a broad

knowledge of various cuisines. They become very good at understanding food in general, but they haven't specialized in anything specific yet.

- **Fine-tuning** is like sending that chef to a specialized restaurant after culinary school. For example, if they go to a high-end Italian restaurant, they will learn to apply their general cooking knowledge specifically to Italian dishes, perfecting pasta making, sauce preparation, and regional specialties. They adapt their broad skills to excel at a particular type of cooking.

In the world of AI:

- **Pre-training** involves training a large language model (LLM) on a massive amount of diverse text data (like the entire internet). During this phase, the model learns general language patterns, grammar, facts, common sense, and how words relate to each other. It becomes a general-purpose language understanding and generation engine.
- **Fine-tuning** involves taking that pre-trained model and training it further on a smaller, specific dataset tailored to a particular task. For instance, fine-tuning a model for sentiment analysis means training it on a dataset of text labeled with sentiments (positive/negative). The model adapts its general language understanding to become very good at *that specific task*.

Business Workflow Design

Let's design a step-by-step business workflow for a customer support scenario using the Generative AI capabilities we've explored.

Scenario: A customer submits a written query or complaint through an online portal or email.

1. **Input Reception:** The customer's text query is received by the system.
2. **Summarization:** The `summarization` pipeline processes the raw customer query to create a concise 'executive summary' of the issue. This helps agents quickly grasp the core problem without reading lengthy texts.
 - **Tool:** Hugging Face `summarization` pipeline (e.g., BART/T5 models).
 - **Output:** A 20-30 word summary of the customer's query.
3. **Sentiment Analysis:** The `sentiment-analysis` pipeline evaluates the sentiment of the original query and/or the generated summary. This helps prioritize urgent or negative customer interactions.
 - **Tool:** Hugging Face `sentiment-analysis` pipeline (e.g., DistilBERT).

- **Output:** Sentiment label (e.g., 'POSITIVE', 'NEGATIVE', 'NEUTRAL') and a confidence score.
4. **Issue Categorization (Optional/Advanced):** Based on keywords in the summary and sentiment, an additional classification model (not covered here but could be a fine-tuned BERT model) could categorize the issue (e.g., 'Billing Inquiry', 'Technical Support', 'Product Feedback').
5. **Agent Interface & Initial Draft Generation:**
- The summarized query, sentiment, and (if applicable) categorization are presented to a customer support agent.
 - Concurrently, a `text-generation` pipeline uses the summary and categorization as a prompt to draft a preliminary response. This response could be a general acknowledgment, a request for more information, or a first attempt at a solution based on common FAQs.
 - **Tool:** Hugging Face `text-generation` pipeline (e.g., GPT-2/GPT-Neo).
 - **Output:** A draft response for the agent to review and refine.
6. **Agent Review & Refinement:** The human agent reviews the AI-generated draft, making necessary corrections, adding personalized touches, and ensuring accuracy before sending it to the customer. This step is crucial for maintaining quality and preventing AI hallucinations.
7. **Customer Response:** The refined response is sent to the customer.

This workflow leverages AI to automate initial understanding and drafting, thereby increasing agent efficiency and reducing response times, while keeping human oversight for critical decision-making and quality assurance.

Hallucination & Bias Discussion

Generative AI models, while powerful, come with inherent risks, particularly **hallucination** and **bias**, which must be carefully considered when deploying them in professional settings.

Hallucination

What it is: AI hallucination refers to the phenomenon where a model generates content that is factually incorrect, nonsensical, or deviates from the provided source material, presenting it as if it were true or accurate. This is not intentional deception but a byproduct of how these models learn patterns and generate text. They predict the next most probable word or phrase based on their training data, which can sometimes lead to creative but inaccurate outputs.

Risks in Professional Settings:

- **Misinformation:** In customer support, a hallucinated response could provide incorrect product information or troubleshooting steps, leading to customer frustration, wasted time, or even damage.
- **Legal/Compliance Issues:** In legal or financial contexts, generating false information could have severe repercussions, including legal liabilities or regulatory fines.
- **Erosion of Trust:** Repeated instances of hallucination can undermine user trust in the AI system and the organization deploying it.
- **Inefficiency:** If agents constantly have to correct hallucinated content, the expected efficiency gains from AI can be negated.

Bias

What it is: AI bias occurs when the model's output reflects or amplifies societal prejudices, stereotypes, or inequities present in its vast training data. Since models learn from human-generated text, they can inadvertently absorb and reproduce existing biases related to gender, race, religion, socioeconomic status, and other sensitive attributes.

Risks in Professional Settings:

- **Discrimination:** In HR applications (e.g., resume screening or candidate recommendations), biased AI could unfairly discriminate against certain demographic groups.
- **Unfair Treatment:** In customer service, biased language could lead to different quality of service or tone depending on perceived customer characteristics, resulting in unfair treatment.
- **Reputational Damage:** Deploying a biased AI system can lead to public outcry, negative press, and significant reputational harm to the company.
- **Ethical Concerns:** Perpetuating or amplifying societal biases through technology is ethically problematic and can exacerbate existing social inequalities.

Mitigating Risks:

- **Human Oversight:** Always keep a human in the loop for critical decision-making and content review, especially for responses that go directly to customers.
- **Diverse Training Data:** Strive to use diverse and carefully curated training data to minimize the introduction and amplification of biases.
- **Bias Detection and Mitigation Tools:** Employ tools and techniques to detect and mitigate bias in model outputs.

- **Regular Auditing:** Continuously monitor and audit AI system performance for signs of hallucination or bias, and retrain/update models as needed.
- **Transparency:** Be transparent with users about when AI is being used and its limitations.

By understanding and actively addressing these risks, organizations can deploy Generative AI more responsibly and effectively.

Generative AI & Transformers Discovery Project

Table of Contents

- [Project Overview](#)
- [Installation](#)
- [Discovery Phase: Setup and Initialization](#)
- [Technical Phase: Implementation](#)
 - [Audit Task: Sentiment Analysis](#)
 - [Creator Task: Text Generation](#)
 - [Executive Summary Task: Summarization](#)
- [Action Phase](#)
 - [Pre-training vs. Fine-tuning Explanation](#)
 - [Business Workflow Design](#)
 - [Hallucination & Bias Discussion](#)
- [Usage](#)
- [Conclusion](#)

Project Overview

This project serves as a hands-on discovery journey into Generative AI and Transformer models, primarily utilizing the Hugging Face `transformers` library. The goal is to set up and experiment with core Natural Language Processing (NLP) tasks like sentiment analysis, text generation, and text summarization. We will explore model capabilities, identify limitations (e.g., sarcasm detection), and understand practical considerations like output length control. Furthermore, the project delves into fundamental AI concepts such as pre-training vs. fine-tuning and considers the broader implications of deploying Generative AI, including potential biases and ethical considerations.

Installation

To run this notebook, you need to install the `transformers` and `torch` libraries. The following command handles the installation:

```
!pip install transformers torch
```

Discovery Phase: Setup and Initialization

In this phase, we set up the environment and initialize the primary Hugging Face pipelines crucial for our exploration:

- **Sentiment Analysis Pipeline:** Used for classifying text into positive or negative sentiments.
- **Text Generation Pipeline:** Used for generating coherent and contextually relevant text based on a given prompt.
- **Summarization Pipeline:** Used for creating concise summaries of longer texts.

These pipelines automatically download and load default pre-trained models (e.g., DistilBERT for sentiment, GPT-2 for text generation, BART-CNN for summarization) and their corresponding tokenizers. Initial `UserWarning` messages regarding `HF_TOKEN` are noted but do not impede the functionality for public models.

Technical Phase: Implementation

Audit Task: Sentiment Analysis

We audited the sentiment analysis pipeline with various test sentences, including a sarcastic one. The model successfully identified straightforward positive and negative sentiments. However, it demonstrated a key limitation: it **failed to correctly detect sarcasm**, misclassifying a sarcastic negative statement as positive. This highlights the challenge of nuanced language understanding for current models.

Detailed Results of 5 Sentiment Analysis Sentences with Scores:

1. "I love this product, it's absolutely fantastic!"
 - Sentiment: POSITIVE (Score: 0.9999)
2. "This service is terrible, I'm extremely disappointed."
 - Sentiment: NEGATIVE (Score: 0.9997)
3. "The weather today is just delightful, isn't it? (said during a heavy storm)"

- Sentiment: POSITIVE (Score: 0.9998) - *Note: This sarcastic statement was misclassified as positive, demonstrating a limitation in detecting nuance.*

4. "It was okay, nothing special."

- Sentiment: NEGATIVE (Score: 0.9821)

5. "I'm so excited to get started on this project!"

- Sentiment: POSITIVE (Score: 0.9997)

Creator Task: Text Generation

The text generation pipeline was used to create domain-specific text based on a customer support prompt. We learned the importance of using `max_new_tokens` instead of `max_length` when attempting to control the length of the generated output, as `max_new_tokens` dictates the number of *new* tokens to generate beyond the prompt, ensuring the desired conciseness.

Experimentation with `num_return_sequences` for Text Generation Coherence:

Prompt: `In the context of customer support, a common issue is user password resets. When a customer requests a password reset, the first step should be to`

1. **Generated Text 1:** `In the context of customer support, a common issue is user password resets. When a customer requests a password reset, the first step should be to verify their identity, often by requesting a one-time password sent to their email or phone number. However, this process can be cumbersome, and customers often forget their password, leading to repeated requests and frustration. To streamline this process, a new approach could`
2. **Generated Text 2:** `In the context of customer support, a common issue is user password resets. When a customer requests a password reset, the first step should be to ask the customer to provide information to verify their identity. This can be a password, date of birth, or other identifying information. However, if the customer has already requested a password reset in the past, the system may remember their request and the previous`
3. **Generated Text 3:** `In the context of customer support, a common issue is user password resets. When a customer requests a`

password reset, the first step should be to verify the customer's identity. This can be done using a combination of methods such as email verification, two-factor authentication, or verifying the customer's account information. Here is a sample Python code that can be used to implement a basic password reset verification process

Coherence Discussion: The texts generated with `num_return_sequences=3` show varying approaches to continuing the prompt. All three provide relevant initial steps for password resets (identity verification). The first two maintain a good conversational flow, expanding on challenges and potential solutions. The third, however, abruptly shifts to suggesting Python code, which, while related to implementation, breaks the narrative coherence expected in a direct customer support context. This demonstrates that while the model can produce syntactically correct text, the *relevance* and *coherence* to the implied conversational context can vary significantly across different generated sequences. Human review is essential to select the most appropriate output.

Executive Summary Task: Summarization